PKM_FORM$

Organización de Lenguajes y Compiladores 1

Proyecto 1

Primer Semestre 2026, Centro Universitario de Occidente

"Id y Enseñad a Todos"

Objetivos generales

- Familiarizar al estudiante con la herramienta JFlex
- Familiarizar al estudiante con la herramienta CUP
- Aplicar conocimientos de análisis léxico y sintáctico.

Objectivos específicos

- Creación de archivos de configuración para JFlex.
- Creación de archivos de configuración para CUP.
- Combinar la funcionalidad de JFlex y Cup en aplicaciones reales.
- Familiarizar al estudiante con el desarrollo de aplicaciones para Android.

Descripción de la actividad

En la actualidad los dispositivos móviles son herramientas necesarias para el desarrollo de las actividades diarias, es importante conocer el proceso de desarrollo de aplicaciones hechas para esa plataforma.
Asimismo, una aplicación que facilite la creación de formularios sería de gran utilidad, ya que permitiría, desde nuestro teléfono, generar una gran variedad de formularios personalizados. Por ello, se propone el desarrollo de una aplicación que permita construir formularios a partir de comandos ingresados por el usuario.

Descripción de la Aplicación

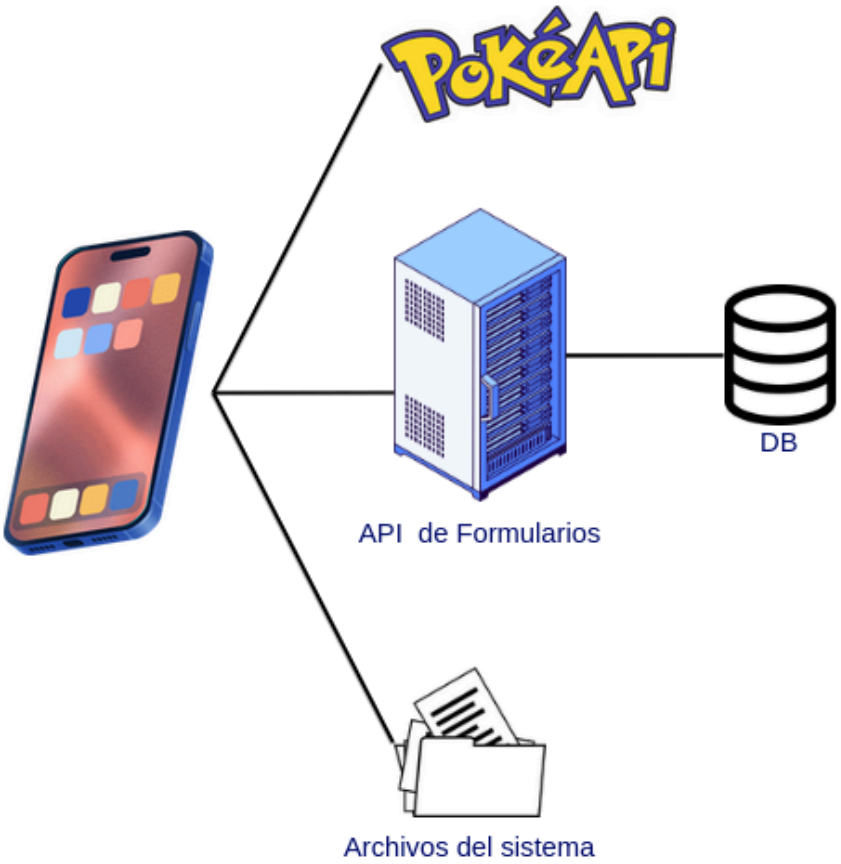
La aplicación debe poder crear formularios a partir de comandos ingresados por el usuario. Los formularios ya creados y procesados se deben poder guardar en un archivo de guardado, así como en un servicio externo para que esté disponible desde un servidor. También se deben poder contestar los formularios cuando el usuario indique que está listo.

Al ver el formulario terminado, se tiene que agregar un botón por default para poder ser enviado. Cuando se “envíe el formulario”, si aplica se debe de mostrar las respuestas correctas, si no, un mensaje de enviado genérico.

Adicionalmente se pide que la aplicación sea amigable con el usuario, contando con ciertas funcionalidades que le permitan ser lo más eficiente posible al crear formularios. Estas características son:

- Guardar los archivos localmente (tanto del lenguaje para la creación de formularios, como el archivo de guardado) así como en un servidor externo.
- Abrir archivos con extensión .form de creación de formularios como el .pkm de persistencia de formularios.
- Reconocer comentarios en el código.
- Que pueda conectarse con una base de datos de terceros para poder tener respuestas predefinidas.
- Si existiera algún error léxico o sintáctico en la ejecución de alguna instrucción se deberá mostrar de la manera más informativa y amigable posible.
- Incluir un mecanismo para agregar emojis al formulario.
- Poder introducir plantillas de código.
- Tener variables para poder tener valores globales.
- Tener un selector de colores personalizado para incluir en el formulario.
- Colorear el código que introduzca el usuario para que le sea más fácil la visualización del código.

Referencia para la interfaz.



Referencia de la arquitectura del sistema

Definición de los Lenguajes

Definición del lenguaje para la creación de formularios

Este lenguaje servirá para ir agregando, o reemplazando todos los elementos del formulario que el usuario aperture o en su defecto uno en blanco. **En la interfaz se tienen que tener ambas opciones, la de reemplazar y la de agregar elementos.**

Este lenguaje es **case sensitive**, por lo que distinguirá entre minúsculas y mayúsculas.

Todos los atributos (por ejemplo width, height y otros) no tienen un orden en específico, pueden venir en cualquier orden, pero no deben repetirse. Todos son obligatorios salvo que se indique lo contrario.

Definición de expresiones

Se consideran expresiones, aquellas que ejecutarán alguna operación aritmética. En las operaciones aritméticas se deben incluir variables de tipo number y string, pero no tipo special.

Operadores aritméticos

Dentro del lenguaje se podrán utilizar, valores numéricos con o sin punto decimal con los cuales se pueden ejecutar operaciones aritméticas básicas las cuales se definen en la siguiente tabla.

Nota: Hay que tener en cuenta la operación unaria, es decir números con signos.

Símbolo	Descripción	Precedencia (de menor a mayor)
+	Suma	1
-	Resta	1
*****	Multiplicación	2
/	División	2
^	Potenciación	3
%	Módulo	3
( )	Paréntesis	4

Operadores de comparación

Se usan para comparar diferentes valores

Símbolo	Descripción	Precedencia (de menor a mayor)
>	Mayor que	0
>=	Mayor o igual que	0
<	Menor que	0
<=	Menor o igual que	0
==	Igualdad	0

!!	Diferente	0

Operadores lógicos

Los operadores lógicos son símbolos o palabras que conectan dos o más expresiones booleanas para formar una nueva expresión.

Símbolo	Descripción	Precedencia (de menor a mayor)
||	o	1
&&	y	1
~	Negación	2

Se pide que en una expresión solo se pueda usar un tipo de operador lógico. Ejemplo:

```
1 > 0 || 1 < 10 || 4 !! 5
1 > 0 && 1 < 10 && 4 !! 5

/*
    Esto produciría un error, pues se están combinando operadores lógicos
*/
1 > 0 && 1 < 10 || 4 !! 5
```

Definición de variables

Se podrán definir variables que ayudarán a tener valores almacenados, según los siguientes tipos:

Tipo	Descripción
number	Almacena números, tanto enteros como decimales.
string	Almacena cadenas de texto.
special	Es un objeto especial que almacena elementos de los formularios y que puede tener comodines

Todas las variables pueden inicializarse y asignarles valores fuera de los elementos. Es decir, no se podrá trabajar con variables dentro de la sintaxis de una sección, una tabla o una pregunta, debe ser fuera.

Number y String

Ejemplo de uso:

```
/*
    Se puede crear la variable y se le deberá asignar un valor por defecto
    para los number es 0
    y para los strings una cadena vacía

*/
number myWidth
string name

/*
    También se puede inicializar de una vez la variable con un valor
*/
number myHeight = 10.0
string lastname = "Hola"

/*
    Posteriormente se le puede cambiar de valor,
    pero hay que tener en cuenta que redefinir la variable no es posible
*/
name = "pokemon"
```

Los siguientes escenarios son errores:

```
$ se volvió a definir una variable
number myVar = 10
number myVar = 20.10

$ se asignó un tipo inadecuado
number myNumber = "hola"
string myString = 516
```

Special

Esta variable permitirá almacenar únicamente preguntas. No se incluyen dentro de este secciones ni bloques de código, tampoco se pueden usar variables dentro de estas. Se usa de la siguiente manera:

```
/*
    Puede ser cualquiera de las preguntas
    que se especifican en la sección correspondiente

    Este tipo de variable si es necesario inicializarla siempre
*/
special myQuestion = OPEN_QUESTION [
    width: number,
    height: number,
    label: number
]

/*
    La variable especial se usa de la siguiente manera
    La invocación de la función por defecto de esta esta
    sirve para agregar lo que contiene al layout
    en este caso la pregunta abierta
*/
myQuestion.draw()
```

Dentro de la sintaxis de una variable special se pueden definir comodines que servirán para poder tener preguntas dinámicas, ejemplo:

```
/*
    Comodín = ?
    Un comodín puede existir dentro de una expresión
    Así como venir suelto
*/
special myQuestion = OPEN_QUESTION [
    width: ?,
    height: 10 + ?,
    label: number,
]
```

```
/*
    Se le deben pasar los comodines a la invocación del método especial draw
    Si no se pasa el número correcto, se debe mostrar un error

    Cuando se muestre la pregunta en pantalla se deben cambiar los comodines
    por las
*/
myQuestion.draw(10, 4)
```

Definición de comentarios

Los comentarios de texto son muy importantes en cualquier lenguaje por lo que podremos definir comentarios en cualquier parte de nuestro archivo, estos comentarios será de una línea y empezaran con el símbolo $ ejemplo:

```
$ esto es un comentario
```

También se podrán tener comentarios de varias líneas:

```
/*
    Esto es un comentario de varias líneas
*/
```

Texto general

Todo el texto irá entre comillas. Aunque dentro de este se podrán agregar emojis, utilizando la sintaxis que se describe a continuación.

Input	Emoji	Notas
@[:)] @[:smile:]	😀	Debe aceptar uno o más paréntesis para la boca.
@[:(] @[:sad:]	🥲	Debe aceptar uno o más paréntesis para la boca.
@[:|] @[:serious:]	😐	Debe aceptar uno o más paréntesis para la boca.
@[<3] @[:heart:]	❤️	Puede aceptar varias veces el símbolo < y también el número 3, pero tiene que seguir el orden.

@[:star:]	⭐	
@[:star:number:] @[:star-number:]	⭐⭐⭐	Permite agregar muchas estrellas a la vez, según el número.
@[:^^:] @[:cat:]	😺	

Ejemplo:

```
"@[:star:2:]@[:<<<<33333:] Hola mundo @[:)))))]@[:heart:]"
```

Output:

```
⭐⭐❤️ Hola mundo 😀❤️
```

Colores

Si no se especifica ningún formato específico o heredado, los colores predefinidos serían negro para las letras y blanco para el fondo.
Los colores se pueden especificar en diferentes formatos:

Formato	Descripción
#FFFFFF	Formato Hex
(10,10,10)	Formato RGB
<10,10,10>	Formato HSL
RED, BLUE, GREEN, PURPLE, SKY, YELLOW, BLACK, WHITE	Colores bases predefinidos

Secciones

Una sección es un contenedor global, se utiliza para mantener organizados nuestros elementos.
Se utilizará la siguiente sintaxis para definir una sección del formulario.

```
SECTION [
      with: number, $ ancho de toda la sección
```

```
height: number, $ alto de toda la sección

$ referencian a la esquina superior izquierda donde empieza la sección
pointX: number,
pointY: number,

/*
    Cómo se irán agregando los elementos, esto es opcional
    Si no se especifica se asume que es VERTICAL

    VERTICAL: De arriba hacia abajo
    HORIZONTAL: De izquierda a derecha
*/
orientation: VERTICAL,

$ La sección de elementos es opcional
elements: {
      $ elementos aquí, separados por comas
      $ no debe haber una coma en el último elemento
},

/*
    Los estilos son opcionales
    y todo lo que se especifica dentro de los corchetes también
    aunque debe venir al menos un estilo, es decir no puede venir vacio
    Estos estilos se heredan a los elementos dentro de la sección,
    salvo que los elementos tengan estilos personalizados
*/
styles [
    "color": color,
    "background color": color, $ color del texto
    "font family": MONO, $ opciones posibles: MONO, SANS_SERIF,CURSIVE
    "text size": number,
    /*
        El número representa el grosor del borde
        tipos posibles: LINE, DOTTED, DOUBLE
    */
    "border": (number, TYPE, color)
]
```

```
]
```

Las secciones se pueden anidar. Ejemplo:

```
SECTION [
      with: number,
      height: number,
      pointX: number,
      pointY: number,

      elements: {
            SECTION [ ... ],
            SECTION [ ... ]
      }
]
```

Resultado:

Tablas

Se pueden definir tablas de la siguiente manera:

```
TABLE [
    width: number,
    height: number,
    pointX: number,
    pointY: number,

    elements: {
```

```
        [ $ primera línea
            {
                $ elemento 1
            },
            {
                $ elemento 2
            },
            {
                $ elemento 3
            }
        ],
        [ $ segunda línea
            {
                $ elemento 4
            },
            {
                $ elemento 5
            },
            {
                $ elemento 6
            }
        ]
    },

    styles [
          "color": color,
          "background color": color,
          "font family": MONO,
          "text size": number,
          "border": (number, TYPE, color)
    ]
]
```

1	2	3
4	5	6

Los elementos de las tablas pueden ser cualquier elemento, una sección, un elemento, una pregunta e incluso otra tabla.
El alto y ancho de la tabla tiene que distribuirse equitativamente.

Elemento de texto

Se podrá definir un elemento que sólo contenga texto y no necesariamente una pregunta, todo texto debe ir dentro de una sección o una tabla, este hereda el alto y el ancho de su padre, sin embargo se podría especificar; la sintaxis es la siguiente:

```
TEXT [
    width: number,  $ opcional
    height: number, $ opcional
    content: "string",

    $ opcional
    styles [
         "color": color
        "background color": color
        "font family": MONO,
        "text size": number
    ]
]
```

Preguntas

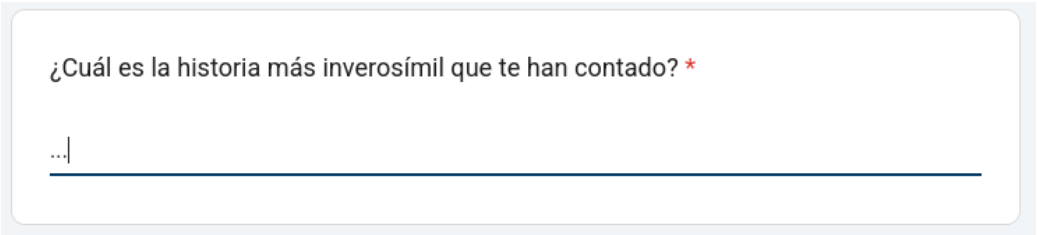
Todas las preguntas tendrán que ir dentro de una sección o una tabla necesariamente, así como el texto, estas heredan ciertos atributos de sus padres, como los estilos el alto y el ancho, aunque se pueden agregar.

Preguntas abiertas

Es una pregunta donde el usuario puede escribir una respuesta. La sintaxis para crearla es la siguiente:

```
OPEN_QUESTION [
    width: number, $ opcional
    height: number, $ opcional
    label: "string",

    $ los estilos son opcionales
    styles [
          "color": color,
          "background color": color,
          "font family": MONO,
          "text size": number
   ]
]
```



Preguntas desplegables

Son aquellas con opciones desplegables.

```
DROP_QUESTION [
    width: number, $ opcional
    height: number, $ opcional
    label: "string",

    options: {"first", "second"},

    /*
       La siguiente especificación es opcional
       Se utiliza para especificar el índice de la respuesta correcta
     se debe mostrar un error si el número se sale del rango de opciones posibles
     */
    correct: number,
```

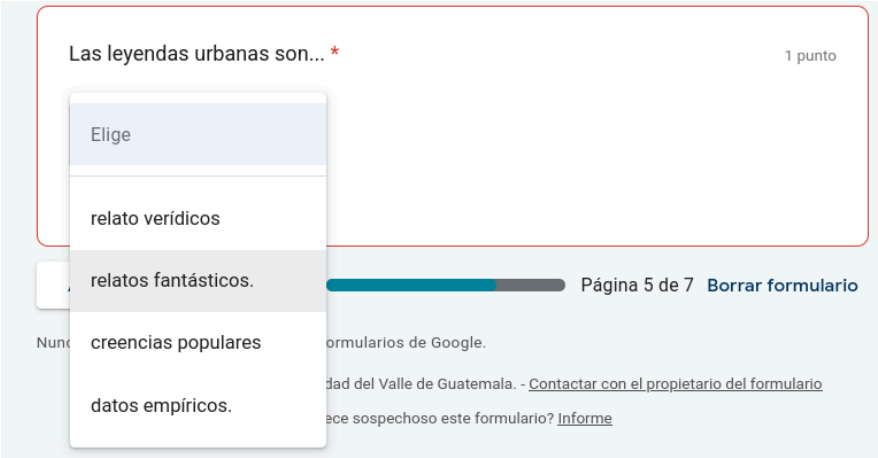
```
    $ los estilos son opcionales
    styles [
          "color": color,
          "background color": color,
          "font family": MONO,
          "text size": number
   ]
]
```

Debido a que muchas veces a los usuarios no se les ocurre ninguna opción, en lugar de especificar las respuestas, se puede ir a extraer un rango de pokemones según su número en la pokedex, desde el rango n hasta el m usando la siguiente función especial:

```
$buscar pokemons según el rango del número en la pokédex
who_is_that_pokemon(NUMBER, 1, 10)
```

Esto se logrará haciendo un REQUEST a la PokéAPI.
Nota: Al guardar las respuestas en el archivo pkm se debe guardar como si el usuario lo hubiera escrito.

Ejemplo de visualización de la pregunta:



Preguntas de selección única

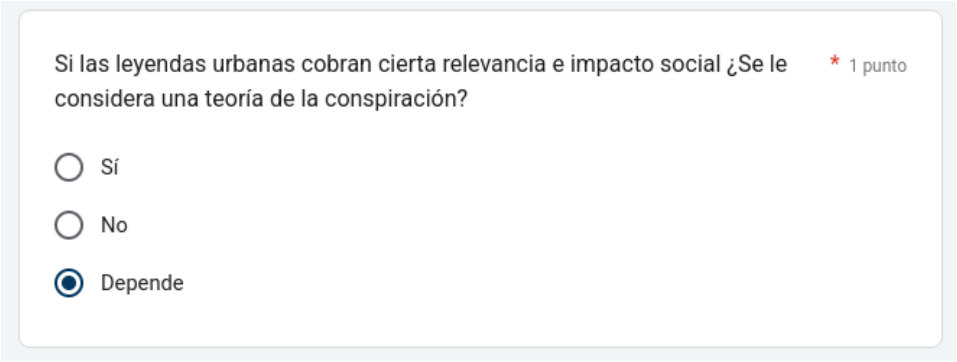
Es una pregunta donde el usuario puede seleccionar solo una respuesta. La sintaxis para crearla es la siguiente:

```
SELECT_QUESTION [
    width: number, $ opcional
    height: number, $ opcional
    options: {"first", "second", "third"},

    $ opcional. Se debe mostrar un error si se sale del rango
    correct: number,

   $ los estilos son opcionales
    styles [
          "color": color,
          "background color": color,
          "font family": MONO,
          "text size": number
   ]
]
```

Si el usuario ingresa más de 5 opciones, se le debe mostrar una advertencia antes de agregar la pregunta al formulario actual.



Preguntas de selección múltiple

Es una pregunta donde el usuario puede seleccionar varias respuestas. La sintaxis para crearla es la siguiente:

```
MULTIPLE_QUESTION [
    width: number, $ opcional
    height: number, $ opcional
```

```
    options: {"first", "second", "third"},

    $ opcional
    correct: {number, number},

    $ los estilos son opcionales
    styles [
          "color": color,
          "background color": color,
          "font family": MONO,
          "text size": number
   ]
]
```

¿De dónde crees que surgen las leyendas urbanas? *

- [ ] De algún chisme.
- [ ] De alguna experiencia real.
- [ ] De un suceso sobrenatural.
- [x] De tecnología malinterpretada.
- [ ] De adaptaciones de leyendas clásicas
- [x] De invenciones literarias o de entretenimiento.

Bloques de código

Son bloques de código que incluyen lógica. Todos los bloques de código se pueden anidar.

Condicional

Se puede usar un condicional para mostrar algunas preguntas solo si se cumple una condición, la sintaxis es la siguiente:

```
IF ( 10 > 10 ) {
    /*
```

```
        Definición o asignación de variables,
        Preguntas
        U otros bloques de código
    */
} ELSE IF ( 10 < 50 ) {
      $ Esta sección es opcional
} ELSE  {
      $ Esta sección es opcional
}
```

Es importante saber que en este caso para que se cumpla la condición la expresión que se evaluará tiene que ser mayor o igual a 1, no se trabajarán con booleanos de 0 y 1, el valor puede ser mayor. Si es 0 o menor el condicional es falso. Esto aplica para todos los condicionales.

Ciclos while y do-while

```
WHILE ( 10 > 10 ) {
      $ instrucciones
}
```

```
DO {
   $ instrucciones
} WHILE ( 10 < 10 || 10 < 10 )
```

Ciclo For

En la parte de la instrucción en el for se asume que se declara la variable (si aún no existe) y se asume también el tipo number.
Si dicha variable ya se ha declarado antes y no es de tipo number, se debe mostrar un error.

```
FOR (i = 0 ; i <= 10 ; i = i + 1 ) {

}
```

Otra forma de usar un ciclo for es la que se muestra a continuación, donde se asume que la variable tiene que ir desde n hasta m. Las reglas de definición y validación de tipos de la variable utilizada aquí son las mismas

```
FOR (i in 1 .. 5) {

}
```

Definición del Lenguaje de guardado pkm

Un formulario ya creado, se puede guardar como archivo con extensión .pkm (la aplicación debe hacerlo), mismo que la aplicación podrá leer más adelante, como es un lenguaje que solo la aplicación leerá no se necesita mostrar errores específicos, sin embargo si este se encuentra manipulado o si no se logra leer se le deberá indicar el usuario, tomando en cuenta que los usuarios pueden manipularlo.

Este lenguaje es **case insensitive,** por lo que no importa si una palabra reservada se escribe en minúscula o mayúscula. En este lenguaje el orden sí es importante.
Los emojis no se deben guardar como tal, se debe usar la notación descrita en la sección de “texto general”.

A continuación se especifica la estructura:

Metadatos

Antes de todo el contenido del formulario se tiene que agregar la siguiente sección, que si bien, se ignorará, es importante tenerla pues servirá como un reporte.

```
###
    Author: Yennifer
    Fecha: 10/10/10
    Hora: 14:10
    Description: ...
    Total de Secciones: número
    Total de Preguntas: número
        Abiertas: numero
        Desplegables: número
        Selección: número
        Múltiples: número
###
```

Estilos

Los estilos se especifican en una etiqueta especial. Los colores siguen la norma que se especificó en la sección correspondiente, pudiendo ser un hex, rgb u otro.

```
<style>
     <color=#FFFFFF/>
```

```
     <background color=<10,10,10> />
     <font family=MONO/>
     <text size=number>
    <border,number,TYPE,color=(10,10,10)/>
</style>
```

Secciones

```
###
     <section=width, height, pointX, pointY, orientation>
    </section>
###
<section=100,10,0,0,VERTICAL>
     <style> ... </style>
     <content>
          # elementos
     </content>
</section>
```

Tablas

```
<table>
     <style> ... </style>
    <content>
        <line>
            <element>

            </element>
            <element>

            </element>
            <element>

            </element>
        </line>
        <line>
            <element>
```

```
            </element>
            <element>

            </element>
            <element>

            </element>
        </line>
    </content>
</table>
```

Texto

```
# <open=width,height,content/>
<open=50,10,"Texto @[:^^:]"/>

<open=50,10,"Texto @[:^^:]">
     # <style> ... </style>
</open>
```

Preguntas

Todas las preguntas se podrán cerrar en la misma etiqueta o con otra etiqueta, sin embargo hay que tomar en cuenta que la etiqueta de estilos puede ir dentro de la misma, para especificar estilos propios.

Preguntas abiertas

```
# <open=width,height,label/>
<open=50,10,"Pregunta abierta @[:smile:]"/>

<open=50,10,"Pregunta abierta @[:smile:]">
     # <style> ... </style>
</open>
```

Preguntas desplegables

Cuando no se cuente con una respuesta correcta, se debe dejar como -1.

```
# <drop=width,height,label,{"option","option"},correct/>
<drop=50,10,"Pregunta desplegable",{"opcion 1", "opcion 2"},-1 />
```

```
<drop=50,10,"Pregunta desplegable",{"opcion 1", "opcion 2"},-1>
     # <style> ... </style>
</drop>
```

Preguntas de selección única

```
# <select=width,height,label,{"option","option"},correct/>
<select=50,10,"Selecciona",{"option 1","option 2"},0/>

<select=50,10,"Selecciona",{"option 1","option 2"},0>
    # <style> ... </style>
</select>
```

Preguntas de selección múltiple

```
# <multiple=width,height,label,{"option","option"},{correct1, correct2}/>

<multiple=50,10,"pregunta",{"option","option"},{}/> $ no hay respuestas correctas
<multiple=50,10,"pregunta",{"option","option"},{10,10}/>

<multiple=50,10,"pregunta",{"option","option"},{10,10}>
     # <style> ... </style>
</multiple>
```

Reporte de errores

Ya sea en una notificación que se pueda cerrar, en un apartado especial para errores u otro; se debe mostrar un reporte de errores para ayudar al usuario a corregir su código, este debe ser similar a lo siguiente:

Lexema	Línea	Columna	Tipo	Descripción
&	2	13	Léxico	Símbolo no existe en el lenguaje
*	3	1	Sintáctico	Se esperaba ‘graficar’ o ‘animar’

Otras funcionalidades

Guardado de archivos

Como se mencionó en la descripción del proyecto, los archivos deben poder guardarse localmente, como archivo .pkm pero también se podrán guardar en un servidor externo.
Al guardarse, de cualquier manera se debe registrar la fecha, la hora y el autor.

En el servidor, se puede usar una base de datos para llevar un registro de los mismos, aunque se solicite que el servidor contenga como tal los archivos .pkm

Ver archivos del servidor

Todos los archivos que se suban al servidor podrán ser accedidos por cualquier persona. Por tanto se pide que exista un apartado en la aplicación móvil, donde se puedan ver todos los formularios subidos. Estos pueden ser descargados en formato .pkm para su posterior edición o solo para contestarlos.
+

Contestar formularios

Cuando se apriete el botón de contestar formularios, se debe mostrar solo la visualización del mismo, ya no el editor.
Existirá un botón en esa vista, cuando se mande el formulario, si aplica se deben mostrar las respuestas correctas, contabilizar la puntuación del usuario; y si no aplica, un mensaje de enviado.

Coloreado del código

Para mejorar la visualización del código, se colorearán ciertos elementos. Esto aplica únicamente para el lenguaje para la creación de formularios, no para el lenguaje de guardado, que nunca se mostrará en la aplicación web. Se tiene la siguiente tabla para referencia:

Elemento	Color
Operadores aritméticos	Verde
Variables	Blanco
Strings literales	Naranja
Números literales	Celeste
Palabras reservadas	Morado
llaves, corchetes, paréntesis	Azul
Especificación de emojis	Amarillo
Otros	Blanco

Inserción de plantillas de código

Se tendrá una opción para insertar código base que luego se pueda editar.

Selector de colores

Para facilidad del usuario se le debe dar la opción de seleccionar el color que quiera para su componente e insertarlo en cualquier formato en el código.

Importante

- La práctica debe ser desarrollada para plataforma Android usando lenguaje de programación Kotlin
- **Usar herramientas JFlex y Cup para cualquier tipo de análisis/proceso léxico y sintáctico.**
- Práctica obligatoria para tener derecho al siguiente proyecto.
- Las copias obtendrán nota de cero y se notificará a coordinación.
- Si se va a utilizar código de internet o Inteligencia Artificial, entender la funcionalidad para que se tome como válido, **su entendimiento del código será evaluada**.

Entrega

Fecha para límite para entregar: 22 de marzo del 2026

Los componentes a entregar utilizando un repositorio git son:

- Código fuente
- Archivo APK
- Manual técnico
    - Detalle de la organización de su proyecto
    - Análisis de gramática para analizador léxico y gramática para analizador sintáctico
    - Diagrama de clases.
- Manual de usuario

Calificación

Pendiente de definir.

Enlaces de Interés

Poké Api